VENDOR-AGNOSTIC REFERENCE BLUEPRINT · ARCHITECTURE, PROTOCOLS & MAPPINGS

The Platform-Agnostic Zero-Trust Agent-to-Database Architecture

Every component, wire protocol, Envoy filter (`ext_authz` / `ext_proc`), cryptographic token, and cross-cloud mapping explained from first principles.

1. Platform-Agnostic Zero-Trust Blueprint (Before Vendor Mapping)

Products like **Vertex AI Agent Engine**, **Google Cloud Agent Gateway**, and **Model Armor** are cloud-specific implementations of deeper architectural primitives. In any production environment—whether you run on Google Cloud, AWS, Azure, or self-hosted Kubernetes—the security of an agent-to-database connection depends on **16 vendor-agnostic building blocks** across **5 trust zones** and **10 cryptographically verified wire hops**.

In the platform-agnostic reference diagram below (**Figure 0A**), **Envoy Proxy** serves as the policy enforcement point at both Ingress (`B1`) and Egress (`B3`), and content safety ("Model Armor") is represented by its underlying architectural pattern: an inline **Envoy External Processor (`ext_proc`) gRPC Guardrail + Deterministic DLP Tokenizer**.

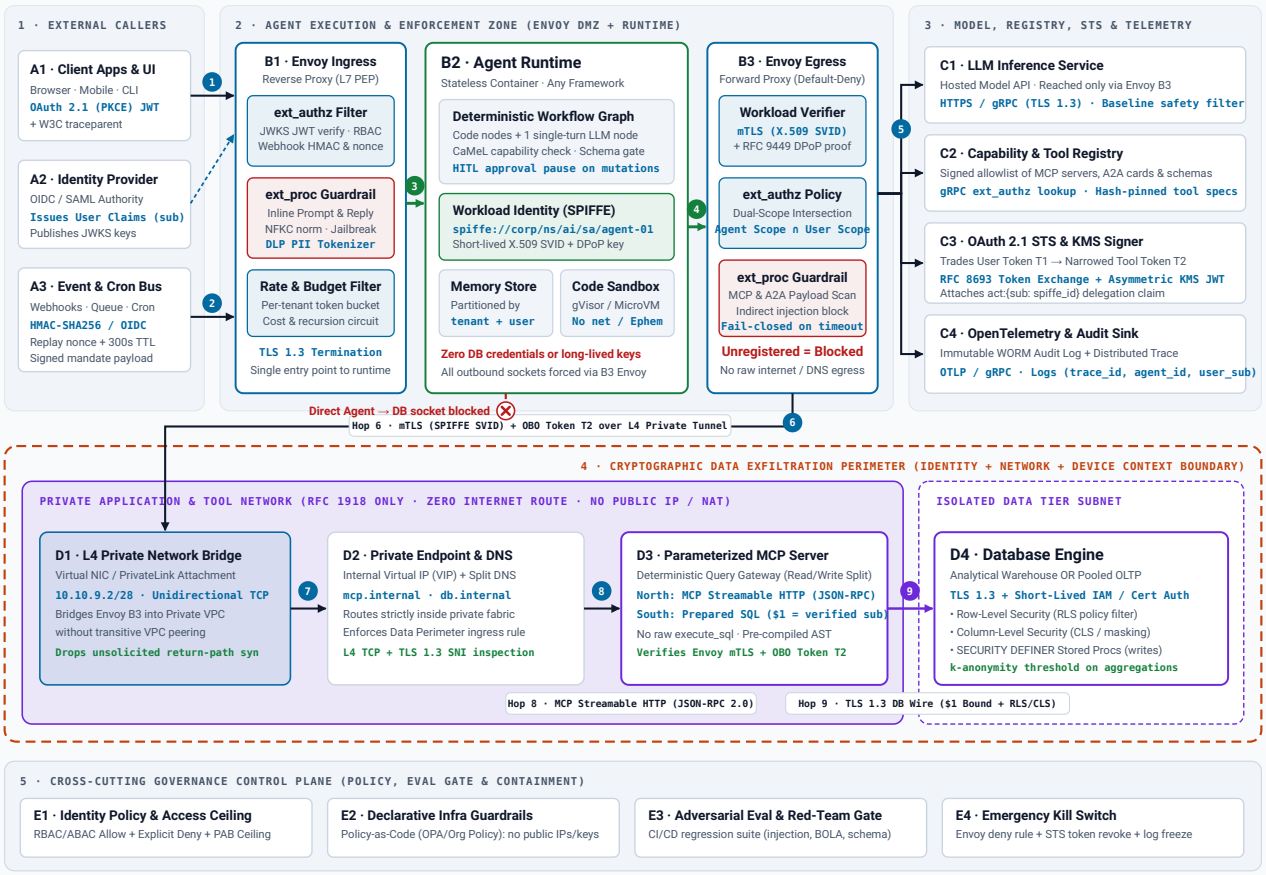


Figure 0A — The Platform-Agnostic Zero-Trust Reference Architecture. Every component is defined by its architectural role, wire protocol, and cryptographic contract rather than a cloud vendor product name.

Envoy L7 Proxies (`PEP`) Inline `ext_proc` Safety Guardrail

Agent Runtime & SPIFFE Identity Private Network, MCP & Database

Cryptographic Data Perimeter

2. Representing "Model Armor" Vendor-Agnostically: Inline Envoy `ext_proc` gRPC Pipeline

In a vendor-agnostic architecture, prompt and tool-output screening is not a magic cloud checkbox—it is an **out-of-process gRPC External Processor (`envoy.filters.http.ext_proc`)** wired into both the Ingress Envoy Proxy (`B1`, screening user prompts and final replies) and the Egress Envoy Proxy (`B3`, screening outbound MCP tool arguments and inbound database rows before they enter the LLM context window).

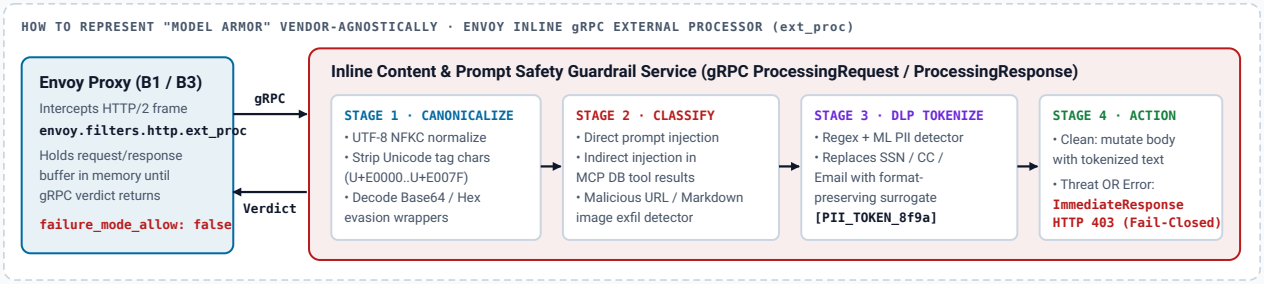


Figure 0A-2 — Vendor-Agnostic Representation of "Model Armor": an Envoy External Processor (`ext_proc`) gRPC service that canonicalizes, classifies, and tokenizes payloads inline with `failure_mode_allow: false`.

3. Wire-by-Wire Connection & Communication Protocol Contract (Hops 1–10)

Every arrow in **Figure 0A** represents a strict network and cryptographic contract between two boxes. If a connection lacks any of the transport, framing, or identity properties listed below, an attacker can bypass the control plane.

HOP	SOURCE → DESTINATION	TRANSPORT & WIRE PROTOCOL	IDENTITY & CREDENTIAL ON THE WIRE	WHAT T RECEIV VERIFY
Hop 1	A1 Client App → B1 Envoy Ingress	HTTPS (TLS 1.3) · HTTP/2 or gRPC-Web · Carries traceparent header	End-user **OAuth 2.1 Bearer / OIDC ID Token** (T1) obtained via Authorization Code + PKCE from A2	Verify TL ext_au JWT sig A2 JW iss , a apply pe bucket r
Hop 1b	B1 Envoy Ingress → A2 Identity Provider	HTTPS (TLS 1.3) · GET /.well-known/jwks.json (cached with short TTL)	Public OIDC Discovery / mTLS client authentication for introspection	Verify Id certificat expecte (iss).
Hop 2	A3 Event & Cron Bus → B1 Envoy Ingress	HTTPS Push (TLS 1.3) · HTTP/2 POST JSON event envelope	**Detached HMAC-SHA256 Signature** (X-Signature-256 + X-Timestamp + X-Nonce) or short-lived Scheduler OIDC JWT	Recomp raw body - times or if no Redis re validate

HOP	SOURCE → DESTINATION	TRANSPORT & WIRE PROTOCOL	IDENTITY & CREDENTIAL ON THE WIRE	WHAT T RECEIV VERIFY
Hop 3	**B1 Envoy Ingress** → **B2 Agent Runtime**	**Pod Localhost Loopback / UDS** (sidecar) OR **mTLS over HTTP/2** (gateway)	Ingress Envoy **SPIFFE X.509 SVID** + Envoy-signed internal header containing verified `user_sub` & `tenant_id`	Agent Ru any conn coming f Envoy's S strips un external receives DLP-toke
Hop 4	**B2 Agent Runtime** → **B3 Envoy Egress**	**mTLS 1.3 over TCP** (or Unix Domain Socket with peer credential check) · `HTTP/2`	**Agent SPIFFE X.509 SVID** (`spiffe://corp/ns/ai/sa/agent-01`) + **RFC 9449 DPoP Proof JWT** + User Token `T1`	B3 Env mTLS, ex (spiffe verifies t header s matches bound (` token.
Hop 5a	**B1 / B3 Envoy** ↔ **ext_proc Guardrail**	**Bi-directional gRPC Stream over mTLS** (`envoy.service.ext_proc.v3.ExternalProcessor`)	Mutual SPIFFE X.509 SVIDs between Envoy and Guardrail service	Envoy se failur false ; ' explicit Proces (`CONTI tokenize `Immedi 403`).
Hop 5b	**B3 Envoy Egress** → **C2 Capability Registry**	**gRPC over mTLS** (`envoy.service.auth.v3.Authorization.Check`)	Tuple: `(agent_spiffe_id, user_sub, target_mcp_uri, tool_name, http_method)`	Evaluate **Intersec** Allowed User RB against s manifest or `PERMIS
Hop 5c	**B3 Envoy Egress** → **C3 OAuth 2.1 STS / KMS**	**HTTPS (TLS 1.3) POST /oauth2/token** (RFC 8693 Token Exchange)	`subject_token = T1` (User JWT), `actor_token = Agent SPIFFE JWT-SVID` , `audience = mcp://db-read` , `scope = orders:read`	STS verif and issu (5-min), restricte **T2** cor user_12 {sub: spiffe: 01} .

HOP	SOURCE → DESTINATION	TRANSPORT & WIRE PROTOCOL	IDENTITY & CREDENTIAL ON THE WIRE	WHAT T RECEIV VERIFY
Hop 5d	**B3 Envoy Egress → C1 LLM Inference API**	**HTTPS (TLS 1.3) / gRPC** · Streaming or Unary inference call	Platform-injected short-lived model-invoker token (attached by B3 Envoy, never held by B2)	Model re DLP-toke approve declarati output is back thr ext_pr reaching
Hop 6	**B3 Envoy Egress → D1 L4 Private Bridge**	**TCP over L4 Private Tunnel Interface** (`10.10.9.2/28`) · Carries outer mTLS 1.3 session	Source IP pinned to dedicated Private Bridge CIDR (`10.10.9.0/28`); carries B3 Envoy SPIFFE X.509 cert	Unidirec B3 → D unsolicit from Dat into Age dropped
Hop 7	**D1 L4 Private Bridge → D2 Private Endpoint & DNS**	**Private L4 Routing (RFC 1918)** + Split-Horizon DNS resolution (mcp.internal → 10.10.2.20)	Network provenance metadata (VPC ID + Bridge Attachment ID) + Caller Identity	**Zone 4 D** verifies t caller ide (`agent/t source n satisfy th Ingress I
Hop 8	**D2 Private Endpoint → D3 Parameterized MCP Server**	**MCP Streamable HTTP (HTTP/2 POST + JSON-RPC 2.0) over mTLS 1.3**	1) Transport: B3 Envoy client X.509 cert 2) Header: Downscoped OBO Token T2 (or KMS-signed Context JWT for async jobs)	D3 veri mTLS ce B3 Env T2 sig == mcp: extracts verifi T2.sub tool args Schema.
Hop 9	**D3 Parameterized MCP Server → D4 Database Engine**	**Mode A (Analytical HTTP):** HTTPS / gRPC Storage API **Mode B (OLTP Pooled TCP):** TLS 1.3 + mTLS over DB Wire Protocol (e.g., pgwire :5432)	**Mode A:** Per-query OAuth 2.1 Bearer Token T2 **Mode B:** Short-lived IAM DB Auth token for toolbox-sa + server-bound parameter $1 = verified_sub (or SET LOCAL app.current_user_id)	Databas compile stateme model-w evaluate **Security Column- (CLS)** fo verifi enforces DEFINER procedu

HOP	SOURCE → DESTINATION	TRANSPORT & WIRE PROTOCOL	IDENTITY & CREDENTIAL ON THE WIRE	WHAT T RECEIV VERIFY
Hop 10	**B1, B2, B3, D3, D4 → C4 OpenTelemetry & Audit Sink**	**OTLP / gRPC over TLS 1.3** + Control-Plane Immutable Log Stream	Service workload identity with write-only (`appendOnly`) permission to WORM log bucket	Correlate under a s tracepa (`trace_i both ac (spiffe subjec (user_s

4. Box-by-Box Specification: What Each Agnostic Component Must Be

Below is the complete engineering specification for every box in **Figure 0A**: its core responsibility, mandatory security invariants, and why substituting a weaker pattern fails.

BOX ID & AGNOSTIC NAME	WHAT IT MUST BE (ARCHITECTURAL ROLE)	MANDATORY TECHNICAL REQUIREMENTS & INVARIANTS	THREATS CLOSED
A1 · A2 · A3 External Callers, IdP & Event Bus	Untrusted entry plane originating human interactive requests and asynchronous system triggers.	• A1 uses OAuth 2.1 Authorization Code + PKCE. • A2 signs short-lived JWTs and exposes a public JWKS endpoint. • A3 signs every webhook/cron payload with HMAC-SHA256 (or OIDC JWT) including a timestamp and unique replay nonce.	T1 Spoofed Caller ASI07
B1 Envoy Ingress Reverse Proxy (PEP)	Dedicated L7 reverse proxy terminating external TLS and enforcing authentication, prompt safety, and rate limits *before* the agent process wakes up.	• **ext_authz filter:** Verifies JWT against A2 JWKS or checks A3 HMAC + Redis replay cache. • **ext_proc filter:** Calls inline content guardrail on user prompt and final agent reply. • **Rate-limit filter:** Enforces per-user and per-tenant token/request budgets.	T1 T4 T6 LLM01 LLM10

BOX ID & AGNOSTIC NAME	WHAT IT MUST BE (ARCHITECTURAL ROLE)	MANDATORY TECHNICAL REQUIREMENTS & INVARIANTS	THREATS CLOSED
`B2` **Agent Runtime Container**	Stateless orchestration container running your agent framework (ADK, LangGraph, custom state machine) with four internal compartments.	• **Workflow Graph:** Deterministic code nodes wrap a single-turn LLM node; enforces Plan-Then-Execute, CaMeL capability tags, and Human-in-the-Loop (`HITL`) pause on writes. • **Workload Identity:** Holds a cryptographic `SPIFFE X.509 SVID` rotated via Workload API. • **Memory Store:** Keyed strictly by `(tenant_id, user_sub)` . • **Code Sandbox:** Runs generated code in an unprivileged gVisor / Firecracker microVM with zero network interfaces. • **Zero-Secret Rule:** Holds *no* database credentials or static API keys.	`T5 Destructive Write` `ASI01` `ASI02` `ASI05` `ASI06`
`B3` **Envoy Egress Forward Proxy (`Default-Deny PEP`)**	Outbound L7 forward proxy (sidecar or shared egress gateway) that intercepts 100% of traffic leaving `B2` .	• **Default-Deny Routing:** Any destination host/URI not in `C2` Registry is rejected with `403` . • **Caller Proof:** Requires `mTLS` (Agent SPIFFE cert) + `RFC 9449 DPoP` proof. • **Dual-Scope Check (`ext_authz`):** Verifies `Agent Scope ∩ User Scope` . • **Inline Payload Scan (`ext_proc`):** Scans outbound MCP arguments and inbound DB rows with `failure_mode_allow: false` .	`T2` `T4` `T7` `ASI03` `ASI04` `LLM02`
`C1 · C2 · C3 · C4` **LLM, Registry, STS & Audit Sink**	Shared control and model services consulted by Envoy Egress (`B3`).	• `C1` **LLM API:** Reached only via `B3` ; receives DLP-tokenized context. • `C2` **Capability Registry:** Cryptographically signed catalog of approved MCP servers and JSON schemas. • `C3` **OAuth 2.1 STS & KMS:** Performs `RFC 8693` On-Behalf-Of (OBO) exchange (`T1 → T2`) and signs Context JWTs with asymmetric KMS keys (`ALLOWED_KIDS`). • `C4` **Telemetry & Audit Sink:** Append-only 400-day WORM log recording `(trace_id, spiffe_id, user_sub, tool_name, row_count)` .	`T1` `T3` `T8` `ASI03` `ASI09`
`D1 · D2` **L4 Private Bridge & Private Endpoint / DNS**	Private L4 network transport connecting the Egress Envoy Proxy into the isolated Data VPC without public IPs or transitive VPC peering.	• `D1` **L4 Bridge:** Dedicated point-to-point virtual NIC (`10.10.9.2/28`) that allows only outbound connections initiated by `B3` . • `D2` **Private Endpoint & Split DNS:** Resolves internal tool and DB hostnames to private RFC 1918 VIPs with zero route to the public internet.	`T7 Network Exfil` `LLM02`

BOX ID & AGNOSTIC NAME	WHAT IT MUST BE (ARCHITECTURAL ROLE)	MANDATORY TECHNICAL REQUIREMENTS & INVARIANTS	THREATS CLOSED

`D3` **Parameterized MCP Tool Server**	Deterministic database gateway speaking MCP (`JSON-RPC 2.0`) on the north side and parameterized SQL on the south side.	• **No Raw `execute_sql` :** Exposes only named, pre-compiled SQL templates (`tools.yaml`).
		• **Out-of-Band Identity Binding:** Extracts `verified_sub` from the validated OBO Token `T2` header (never from LLM arguments) and binds it to SQL parameter `$1` .
		• **Read/Write Split:** Runs separate read-only and write-scoped service instances.
			`T2 Arbitrary SQL`
			`T3 Cross-Tenant Read`
			`A03:2021 SQLi`

`D4` **Database Engine (OLTP / Warehouse)**	Final authoritative data store enforcing row, column, and mutation invariants inside the database kernel.	• **Row-Level Security (RLS):** Per-request OAuth `SESSION_USER()` filter (stateless HTTP warehouse) or `SET LOCAL` + `FORCE ROW LEVEL SECURITY` / Parameterized Secure Views (stateful pooled TCP OLTP).
		• **Column-Level Security (CLS):** Policy tags / column `GRANT` s + `security_invoker = true` views.
		• **Bounded Writes:** Mutations only via atomic `SECURITY DEFINER` stored procedures.
		• **Aggregation Floor:** `HAVING COUNT(DISTINCT owner_id) >= 5` (`k`-anonymity).
			`T3 BOLA / IDOR`
			`T5 Destructive Write`
			`API1:2023 BOLA`

`Zone 4 & E1–E4` **Data Perimeter & Control Plane**	Outer cryptographic data boundary and continuous governance controls across all environments.	• **Zone 4 Data Perimeter:** Rejects any data request unless *both* caller identity and source network (`D1`/`D2`) belong to the perimeter (stops stolen tokens used from outside). • `E1–E4` **Control Plane:** Principal Access Boundary (`PAB`), declarative Org Policies (`OPA`), CI/CD adversarial eval gate, and 3-step emergency Kill Switch.	`T7` `T8` `ASI08` `ASI10`

5. Cross-Platform Implementation Mapping (Agnostic → Open-Source / K8s → Google Cloud → AWS → Azure)

Once you lock in the agnostic architecture in **Figure 0A**, mapping it to any target platform is a mechanical 1-to-1 substitution:

AGNOSTIC COMPONENT (FIGURE 0A)	OPEN-SOURCE / SELF-HOSTED K8S	GOOGLE CLOUD IMPLEMENTATION (FIGURE 0B)	AWS EQUIVALENT	AZURE EQUIVALENT
`B1` **Envoy Ingress Proxy (`PEP`)**	**Envoy Proxy** / Istio Ingress Gateway + OPA `ext_authz` + Redis rate limit	**Agent Gateway (`CLIENT_TO_AGENT`)** + Cloud Armor + Identity-Aware Proxy (IAP)	Envoy on ECS/EKS + AWS WAF + API Gateway / ALB OIDC Authorizer	Envoy on AKS / Azure Application Gateway + Front Door WAF + Entra EasyAuth
`B1/B3` **Inline `ext_proc` Safety Guardrail**	**Envoy `ext_proc` gRPC filter** → Llama Guard / NeMo Guardrails + Microsoft Presidio DLP	**Model Armor** (inline Service Extension / Floor Settings) + **Cloud SDP (DLP)**	Envoy `ext_proc` → Amazon Bedrock Guardrails ApplyGuardrail API + Macie/Comprehend PII	Envoy `ext_proc` → Azure AI Content Safety + Prompt Shields + Purview DLP
`B2` **Agent Runtime & Workload ID**	Container on K8s + **SPIFFE / SPIRE** (`X.509-SVID`) + LangGraph/ADK + gVisor (`runsc`)	**ADK 2.x on Vertex AI Agent Engine** (or Cloud Run) + **Agent Identity (SPIFFE)** + Code Execution Sandbox	Bedrock AgentCore / EKS Pod + SPIRE / IAM Roles for Service Accounts (IRSA) + Firecracker	Azure Container Apps / AKS + Workload Identity Federation + Dynamic Sessions Sandbox
`B3` **Envoy Egress Proxy (`Default-Deny`)**	**Envoy Sidecar / Egress Gateway** (`mTLS` + `DPoP` + OPA `ext_authz` + NetworkPolicy)	**Agent Gateway (`AGENT_TO_MCP` / `AGENT_TO_AGENT`)** or **Cloud Run Envoy Sidecar** + IAP	Envoy Egress Sidecar + AWS VPC Lattice / App Mesh + Verified Permissions (Cedar)	Envoy Egress Sidecar + Azure API Management (Self-Hosted Gateway) + NSG Default-Deny
`C2` **Capability & Tool Registry**	Signed GitOps YAML Catalog / Backstage + OPA bundle server	**Google Cloud Agent Registry** (approved agents, MCP servers & schemas)	AWS Service Catalog / Private API Registry + Systems Manager Parameter Store	Azure API Center (MCP & Agent Inventory)
`C3` **OAuth 2.1 STS & KMS Signer**	**Keycloak / Ory Hydra** (`RFC 8693` STS) + HashiCorp Vault Transit KMS	**Agent Engine Auth Manager** + **Cloud IAM STS** + **Cloud KMS** (`AsymmetricSign`)	AWS STS / Cognito + AWS KMS (`Sign` API with ECC_NIST_P256)	Microsoft Entra ID On-Behalf-Of (OBO) flow + Azure Key Vault HSM
`D1 · D2` **L4 Private Bridge & Endpoint**	WireGuard / Cilium ClusterMesh L4 Point-to-Point Tunnel + CoreDNS Split-Horizon	**Private Service Connect (PSC) Network Attachment (`10.10.9.2`)** + **PSC Endpoint** + Private Cloud DNS	AWS PrivateLink Interface Endpoint + VPC Endpoint Service (NLB) + Route 53 Private Hosted Zone	Azure Private Link Service + Private Endpoint + Azure Private DNS Zone
`Zone 4` **Cryptographic Data Perimeter**	SPIFFE Federation + Cilium L7/L4 Identity-Aware Network Policy + mTLS mesh boundary	**VPC Service Controls (VPC-SC)** Perimeter (`bigquery.googleapis.com`, `sqladmin.googleapis.com`, `run.googleapis.com`)	AWS Data Perimeter (SCPs + VPC Endpoint Policies `aws:PrincipalOrgID` + `aws:SourceVpce`)	Azure Network Security Perimeter (NSP) + Conditional Access for Workload Identities
`D3` **Parameterized MCP Tool Server**	**MCP Toolbox for Databases** (OSS binary) or Custom FastMCP Server with pre-compiled SQL	**MCP Toolbox for Databases on Cloud Run** (`--ingress=internal`, `tools.yaml`, `authServices`)	MCP Toolbox on AWS Fargate (Private Subnet) with RDS Proxy IAM Auth	MCP Toolbox on Azure Container Apps (Internal VNet) with Managed Identity

AGNOSTIC COMPONENT (FIGURE 0A)	OPEN-SOURCE / SELF-HOSTED K8S	GOOGLE CLOUD IMPLEMENTATION (FIGURE 0B)	AWS EQUIVALENT	AZURE EQUIVALENT
`D4` **Database Engine (`RLS` / `CLS`)**	**PostgreSQL 16+** (`FORCE ROW LEVEL SECURITY`, `SET LOCAL`, `security_invoker` views, `SECURITY DEFINER` procs)	**BigQuery** (Stateless HTTP + Policy Tags + RLS) & **Cloud SQL / AlloyDB** (Stateful TCP + Parameterized Secure Views / PG RLS)	Amazon Redshift RLS/CLS & Amazon Aurora PostgreSQL (`FORCE RLS` + `SECURITY DEFINER`)	Microsoft Fabric / Synapse RLS/DDM & Azure Database for PostgreSQL (`FORCE RLS`)